Bridging the Gap: Implementing Binary Splitting Algorithms on Constrained Mobile Hardware

Author: LgK-S-Dev

Program Name: Simple Pi

1. Introduction:

On 2026.03.03-05 my school put up some decoration due to the close date the Pi day, so after me helping to put some of these decorations up, I joked to my friend that " I will calculate 1 billion digits of Pi and send it to my math's teacher on international Pi day itself to se her reaction. So, as you may see it all started as a joke, but before those days I had a less efficient and overall worse, but still good enough C++ code I ran on an older phone I had and though "Maybe I can use this." So, I did, I don't think it is relevant what that old code did, mostly it calculated prime, twin primes and perfect numbers. Due to the nature of these numbers, they could be calculator relatively fast on a phone. So, I copied the code to my new phone and started working. At first it was hard; I failed a lot but then after some research I found out about the Chudnovsky Algorithm. I researched all day, mostly at night, and made the simplest form of it. It worked slowly and overheated the phone. So, after trial and error I finally found out about binary splitting, I learnt the basics which led to multiple unfortunate errors and logging problems but after more trial and failing all day long I finally got it. I made the formula of code I used it was still flawed but hopefully usable most of the time. But enough about how I got here, while I was doing my research, I reached a point where I found out about how hard it is to calculate Pi, and why it is used in countless benchmarking programs. It is CPU, RAM, and I/O heavy if done incorrectly. I lived through the last problem. First, I was storing all the digits repeatedly all the time. So, I had the first 10 thousand digits, the 10 again with another set of ten and so on and so on . That lead to about 1-2 Gigabytes of data that it took to store only the first 10 million digits. It was before I optimized most of the code, I hopefully caught on. Then that got me thinking, wouldn't this fill my whole storage over a few billion digits, which at the time I thought would be achieved in an hour by a fast algorithm. Which may be true for specialized desktops with programs written by professionals, I had neither. So I came up with GMP State Serialization. Basically, explained just keeping the least amount of data in binary for as I mentioned earlier. At this point it was not just a joke; I took it seriously. So, I had to continue. I thought for tomorrow too, I implemented automatic compression with checkpoints to failsafe the whole thing.

The next few questions as you read this if you know at least 1/3 of what I wrote is that either, how did a 13-year-old do this? Which I don't even have the answer to; Or that how does a phone run this? Which the answer is not great, not terrible.

So I had a goal, the goal being at least that billion and I will go with pride I though I had a very flawed project that is the equivalent of a bicycle being held together by duct tape. Which works until it does not.

So let's say a few things about me, if you will. I am as I mentioned earlier and as you may see 13-year-old me (my name is not relevant), a seventh grader. As previously mentioned, I am a 13-year-old seventh grader. Initially, I found it hard to believe that the algorithms I was implementing—Binary Splitting and Chudnovsky—are the exact same ones used by world-record-breaking software like y-cruncher. This project proves that high-performance computing is not limited to professional labs; with enough dedication, it is accessible to students on consumer hardware.

Methodology:

1.1 THE ALGORITHM

I used the Chudnovsky Algorithm, which is currently the fastest known method for calculating Pi. It converges at a rate of about 14.18 digits per term. It uses advanced math to calculate Pi itself, it is very hardware heavy if not optimized!

1.2 the primary optimization technique: *Binary splitting*

Calculating large numbers, advanced problems millions of times in a second is slow and hardware heavy. That was one of the biggest drawbacks I had to face. My solution was what experts would call Binary Splitting. Binary Splitting is breaking the math into chucks and then computing them in order, a less formal way is saying it is the "Divide and Conquer piece by piece". Think of it like a war, where the evil numbers (Pi) is being invaded by let's say Chudnovsky and their kingdom. The kingdom's name is our program which I call Pi Theory or Simply Pi. Binary splitting is like going strategically dividing the Evil Numbers kingdom into multiple pieces and going piece by piece. Like breaking a big territory into smaller ones, so it is faster to attempt and successfully take over that smaller territory. The only difference is that in The Evil Numbers Kingdom there isn't a kind of force like an army to hold enemies back but harsh terrain, big walls, and powerful people in charge and powerful people in defending their kingdom if the kingdom is overtook. But of course, Chudnovsky and the new kingdom's leaders show that the old kingdom was bad and that they get improvements or something so that people come and finally let and that territory Is finally claimed successfully by our program. Think of binary splitting and our whole code like this. To invade an entire kingdom at one is very hard and needs special hardware, or at least a desktop. This applies to our calculations too, big calculations are hard, multiple smaller ones are faster. If we process then recursively then we will be able to avoid something call bottleneck, which is due to our

limited and standardized hardware, and our not the most professional software. It allows us to move faster while doing the same load in smaller chunks and then stitching them together in the end. It is also RAM heavy!

1.3 the storage problem: *GMP State Serialization*

So as i mentioned earlier I am using GMP State Serialization which is a way to make numbers take up less space in very basic terms. What happens is that if I would have kept just storing all the digits as it is that would make considerably bigger problems as time goes on:

- It would make storing the digits considerably worse, it would have so that a single text file would reach over 1 gigabytes of data, which would correspond to about 1 billion digits. Which Is unusable for most text editors, or text file readers.
    - One digit of text is about 1 byte if we store it in ASCII text
- It would slow the process down, since after about 10-50 million digits the conversion only take minutes if not hours because as we know computers use binary, which needs to be calculated back into readable text. I am as of writing this line have about 493 million (to be exact it is 493521332) digits, the time it would take to convert would be worse than my solution.
- If coded incorrectly it would save the whole at that point calculated whole number which I found out that it can when it grew rapidly in the early 10-20 million digits. If I didn't notice I probably wouldn't be at half of what the current digits are

My solution was storing GMP's (a way to calculate advanced and very high numbers which otherwise would be borderline impossible. It also uses pure binary to perform these calculations.) raw binary instead of text. This is far more efficient and faster, due to the reasons listed above because this way I was able to go around these setbacks and preform the calculations faster and efficiently.

1.4 The fail safe: *'Checkpoints'*

So, as I will explain in greater detail, a phone has very little space to work with and because of this heat can build up and thermal throttling begin. Also, because of how expensive each calculation is, it is very hardware heavy, this can mean that the phone runs out of battery and or crash due to limitations. This can lead to corruption, and to the fact that the program loses where it was last time and also due to this reason if the user would stop the program it would start over, leading to wasted time and calculations as well as double writing. I came up with the idea to save a kind of state to save where each the state is the GMP's way of telling itself where it is and what calculations to perform. This way if the user or unforeseen circumstances stop the program at the restart it knows where it was regardless if the file it is saved into is in binary or normal text, at this time it does not matter.

With the circumstances I found out that while I primarily purposed this for general power loss or crashes it is also applicable as a defense against something called informally 'The Silent Data Corruption' which can be theoretically be caused by hardware fault and or unforeseeable circumstances like cosmic radiation and or severe overheating that can corrupt and or slightly set off a domino effect that a small miscalculation can lead to severe circumstances like the invalidity of the entire project. By creating saving point we can easily identify what parts are invalid or corrupted if they are and can revert back to a point where they are not. This has not been done and is only theoretical possible and applicable.

1.5 The multiplier: *multi-threading*

This part is very important since with threading you can far double the speed of the calculations. A CPU is a component of almost anything. It's in the phone I used too, my CPU had 8 cores. 2 2.00 Ghz, and the rest is 1.8 Ghz. After discovering and successfully implementing multi-threading I had to do a simple benchmark too, that I will use the data collected to use. The benchmark

- Ran for the desired amount of time set by the user in the code
- Ran to the set amount of cores for the set amount of time
- It went by the first core for let's say 60 seconds, the 2 cores for 60 seconds and so on and so on
- The benchmark use the same algorithm as the program I wrote that we are talking about, it calculated Pi
- The only difference is that it doesn't create any permanent Pi numbers
- It only generates the desired benchmark in a text file each time as ran
- It calculated many things and helped me throughout everything to tweak the settings, for the cores is one example.
- It was used to make this paper as mentioned earlier, as for data and the code itself

1.6 : The Bottleneck

To improve performance, console output (printf) was kept to a minimum. Printing text to the screen creates an I/O bottleneck that can slow down the program. By updating the status only once every few million terms, most CPU time remained focused on the actual calculations. This means that the slow printing is not often so the CPU can do the arithmetic calculations in the meantime.

2. Challenges & limitations:

2.1 The heat: *Thermal Throttling*

Heat management was a major limitation I have had to face. First of all, let's get everyone in the clear. A phone is small, we can agree that compared to a desktop or a laptop It is in fact very small. It also doesn't use fans or water cooling to dissipate the heat, it uses passive cooling which means it just relies on the air and the surface that it is on to get the heat off of the phone. It is not efficient for long use cases. The phone detects and throttles (slows down) the components to let it cool down, this is not good for our program, but it is for the greater good, if it were not to do that. The phone could be damaged and that would be unfortunate because that would render the phone unusable. So I had to calculate for that, time to time because of thermal throttling it slows down. Of course, I cannot change nor want to change it.

2.2 The solution to heat: *Al (aluminium)*

I talked about the problem, heat. I noticed this multiple times, when I left the phone calculating overnight it slowed down, and showed near zero or at least lower speeds. I got a solution that I thought of. I will explain that now:

1. *prototype*: First I got my hands on some aluminum foil. I folded it in half, placed it under the phone. I also folded the sides upwards so that it will dissipate more heat. The Why to aluminium, because as I know it. It is a very good heat conductor, It is also widely used in heat sinks and heat exchangers. I also folded the sides of the aluminum up for more surface area to be in contact with the air so better heat dissipation is achieved.
2. *prototype:* This introduced active cooling. I used a small fan with a 9V battery and a breadboard with an external PS that I taken out of an Arduino set that I owned. The setup looked like this: 9V battery → Power supply external input → breadboard (with the 5 V and grounding pins) → small fan.
3. *prototype:* This prototype changed a few components, but the cooling didn't change. I changed the 9V battery and the power supply too to my mother's power bank and the Arduino itself. This way I didn't have to worry about the battery discharging, and me being able to recharge the battery, in this context the power bank. The setup looked like this: Power bank → Arduino → breadboard (via the 5V and the ground onboard pins) →small fan directed at the aluminum foil and the phone

4.prototype: The final improvement was raising the aluminium foil structure slightly above the surface it was placed on. This allowed air to flow both above and below the aluminium, improving overall heat exchange and making the cooling system more effective.

Final diagram:

Fan

↓

[**airflow** ***]***

↓

Phone

↓

Aluminium foil heat spreader

↓

Air gap

↓

Table surface

2.3 **The two biggest restraints:** ***The battery and stability***

As we may know, no energy is gone, it just becomes another kind. This time from electric power it becomes heat, this heat is the main cause of the previous problem listed. Also a normal phone which I used is not designed to run on nearly 100% almost all the time for days on end. Because of that stability is a question too, because if a battery is drained faster than if the code is nor run. Of course this was calculated for after I got to know about this, fortunately this was never a problem, that was unmanageable currently as of writing this time. It is noticeably faster in using the battery, but it can still last hours with usage while running. This can also lead to crashes in the code, the complier or android itself can run into a problem too, and it may also stop the complier if it detects that it is unwanted battery heavy. Which it is, it used *Battery used by app: 62.18%. CPU total:8h 9m 6s. CPU foreground: 3h 22m 35s 2026.03.06 (This is an estimation, not an exact number. It is not completely untrue but may be differential in the minutes or second, maybe more)*

2.4 Intermittent Execution Constraint:

Unlike a dedicated desktop server, the mobile hardware was required for daily use. This introduced the challenge of irregular execution. The calculation had to be paused and resumed daily, putting the checkpoint system under extensive testing. Over a period of {waiting} days, the program was stopped and restarted approximately {waiting} times without data corruption, proving the robustness of the serialization method and the checkpoint system.

2.5 The hardware limitations

I had a fairly good subject as in my phone. It's specifications will be listed in a moment after this. Due to the limited but mid-range amount of RAM I had to be careful but not to

the exact point. I also had to think of my storage, it is limited too. Though only by modern standards. It was and is an about 2021 midrange phone with the specifications listed below.

3. **Hardware specifications**

Device: Xiaomi Redmi 12 (23053RN02Y)

CPU: MediaTek Helio G88 2.00Ghz

RAM: 8.00 + 4.00 gigabytes

storage: 256 gigabytes

OS: Android 15, Xiaomi HyperOs 2.0.205.0 (probably updated since writing this)

Programming Language: C++

Complier: Cxxdroid (complier)

Library:

- cstdio
- cstdlib
- cstdint
- cstring
- cerrno
- ctime
- chrono
- thread
- vector
- atomic
- mutex
- fstream
- sstream
- iomanip
- iostream
- GMP
- zlib

4. **Results: The results are currently pending. I have not yet finished the one billion milestone. It will be update with the conclusion as well.**

5. **Conclusion: The conclusion is currently pending. I have not yet finished the one billion milestone. It will be updated with the results as well.**